嵌入式系统实验指导书

试验 3

2025 年 9 月

目录

1. 实验三：传感器实验

1.1. 温湿度传感器实验

1.1.1. 实验目的

1） 了解温湿度传感器的原理和使用

2） 掌握通过 STM32 读取温湿度数据，并通过串口和 OLED 屏显示出来的方法

1.1.2. 实验环境

1) 硬件：STM32 开发板，OLED 显示模块，程序下载调试板，ARM JLINK 仿真器，USB 方口线，PC 机，Micro-USB 线/交叉串口线，温湿度传感器模块，DC 5V 电源。

图 1-1 STM32 开发板

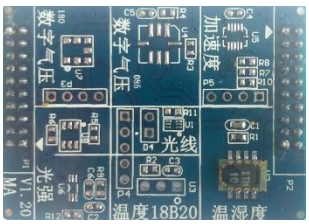


图 1-2 温湿度传感器模块

图 1-3 OLED 显示模块

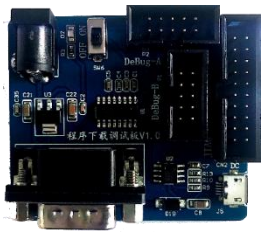


图 1-4 程序下载调试板

图 1-5 ARM JLINK 仿真器

2) 软件：Windows7 及以上系统，Keil5 MDK-ARM5.43a，串口调试助手 SuperCom。

1.1.3. 实验原理

SHTxx 系列单芯片传感器是一款含有已校准数字信号输出的温湿度复合传感器。它应用专利的工业 CMOS 过程微加工技术（CMOSens®），确保产品具有极高的可靠性与卓越的长期稳定性。传感器包括一个电容式聚合体测湿元件和一个能隙式测温元件，并与一个 14 位的 A/D 转换器以及串行接口电路在同一

芯片上实现无缝连接。因此，该产品具有品质卓越、超快响应、抗干扰能力强、性价比极高等优点。每个 SHTxx 传感器都在极为精确的湿度校验室中进行校准。校准系数以程序的形式储存在 OTP 内存中，传感器内部在检测信号的处理过程中要调用这些校准系数。两线制串行接口和内部基准电压，使系统集成变得简易快捷。超小的体积、极低的功耗，使其成为各类应用甚至最为苛刻的应用场合的最佳选则。产品提供表面贴片 LCC（无铅芯片）或 4 针单排引脚封装。

温湿度取值原理如下：

1) 发送命令

用一组“启动传输”时序，来表示数据传输的初始化。它包括：当 SCK 时钟高电平时 DATA 翻转为低电平，紧接着 SCK 变为低电平，随后是在 SCK 时钟高电平时 DATA 翻转为高电平，如图 1-6 所示。

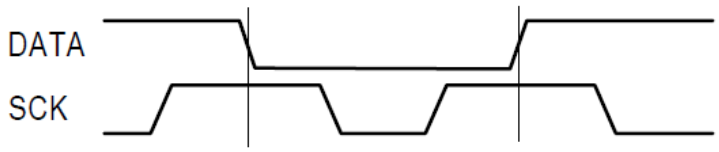


图 1-6 启动时序图

后续命令包含三个地址位（目前只支持“000”）和五个命令位。SHTxx 会以下述方式表示已正确地接收到指令：在第 8 个 SCK 时钟的下降沿之后，将 DATA 下拉为低电平（ACK 位）。在第 9 个 SCK 时钟的下降沿之后，释放 DATA（恢复高电平）。

2) 测量时序(RH 和 T)

发布一组测量命令（‘00000101’表示相对湿度 RH，‘00000011’表示温度 T）后，控制器要等待测量结束。这个过程需要大约 20/80/320ms，分别对应 8/12/14bit 测量。确切的时间随内部晶振速度，最多可能有-30%的变化。SHTxx 通过下拉 DATA 至低电平并进入空闲模式，表示测量的结束。控制器在再次触发 SCK 时钟前，必须等待这个“数据备妥”信号来读出数据。检测数据可以先被存储，这样控制器可以继续执行其它任务，在需要时再读出数据。接着传输 2 个字节的测量数据和 1 个字节的 CRC 奇偶校验。uC 需要通过下拉 DATA 为低电平，以确认每个字节。所有的数据从 MSB 开始，右值有效（例如：对于 12bit 数据，从第 5 个 SCK 时钟起算作 MSB；而对于 8bit 数据，首字节则无意义）。

用 CRC 数据的确认位，表明通讯结束。如果不使用 CRC-8 校验，控制器可以在测量值 LSB 后，通过保持确认位 ack 高电平，来中止通讯。在测量和通讯结束后，SHTxx 自动转入休眠模式,如图 1-7 和图 1-8 所示。

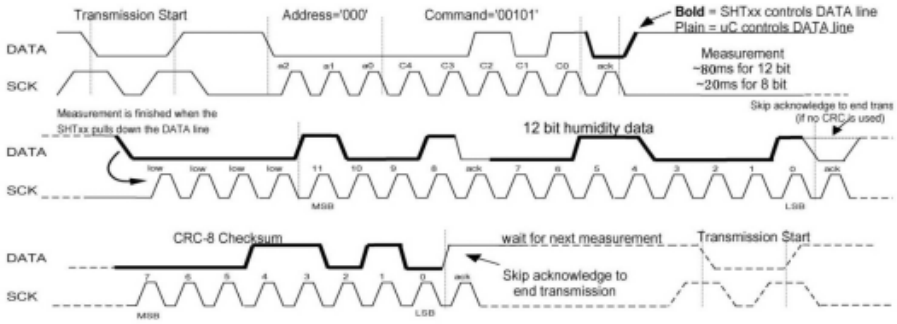


图 1-7 时序图

测量时序举例：“0000’1001 ’0011’0001”=2353=75.79%RH(未包含温度补偿)。

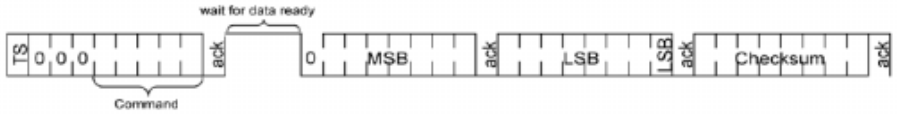


图 1-8 测量时序概览（TS=启动传输）

3) 通讯复位时序

如果与 SHTxx 通讯中断，下列信号时序可以复位串口：当 DATA 保持高电平时，触发 SCK 时钟 9 次或更多。在下一次指令前，发送一个“传输启动”时序。这些时序只复位串口，状态寄存器内容仍然保留，如图 1-9 所示。

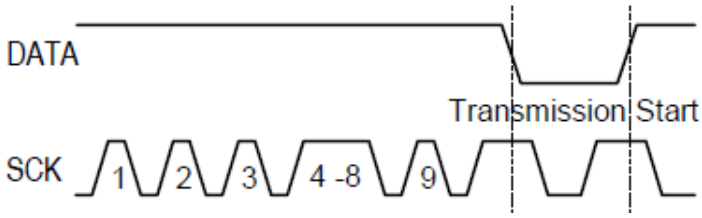


图 1-9 通讯复位时序

4) 相对湿度

为了补偿湿度传感器的非线性以获取准确数据，建议使用如下公式修正输出数值：

$RH_{linear}=c1+c2\bullet SO_{RH}+c3\bullet SO_{RH}^2$

对高于 99%RH 的那些测量值则表示空气已经完全饱和，必须被处理成显示值均为 100%RH。湿度传感器对电压基本上没有依赖性，如图 1-10 所示。

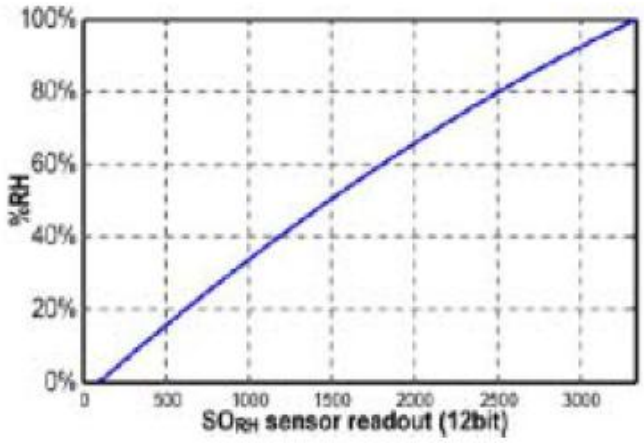


图 1-10 RH 转换响度湿度

实际测量温度与 25℃ (~77℉)相差较大时，应考虑湿度传感器的温度修正系数，如图 1-11 所示，公式如下：

$RH_{true} = (T_{°C} - 25)•(t_1 + t_2 •SO_{RH}) + RH_{linear}$

SO_{RH}	c_1	c_2	c_3
12 bit	-4	0.0405	$-2.8 * 10^{-6}$
8 bit	-4	0.648	$-7.2 * 10^{-4}$

图 1-11 湿度转换系数

由能隙材料 PTAT (正比于绝对温度) 研发的温度传感器具有极好的线性。可用如下公式将数字输出转换为温度值，温度转换系数如图 1-12 所示：

$Temperature = d_1 + d_2 •SO_T$

VDD	d_1 [℃]	d_1 [℉]
5V	-40.00	-40.00
4V	-39.75	-39.55
3.5V[3]	-39.66	-39.39
3V[3]	-39.60	-39.28
2.5V[3]	-39.55	-39.19

SO_T	d_2 [℃]	d_2 [℉]
14bit	0.01	0.018
12bit	0.04	0.072

图 1-12 温度转换系数

在极端工作条件下测量温度时，可使用进一步的补偿算法以获取高精度。

温湿度模块与 STM32 的接口电路如图 1-13 所示。

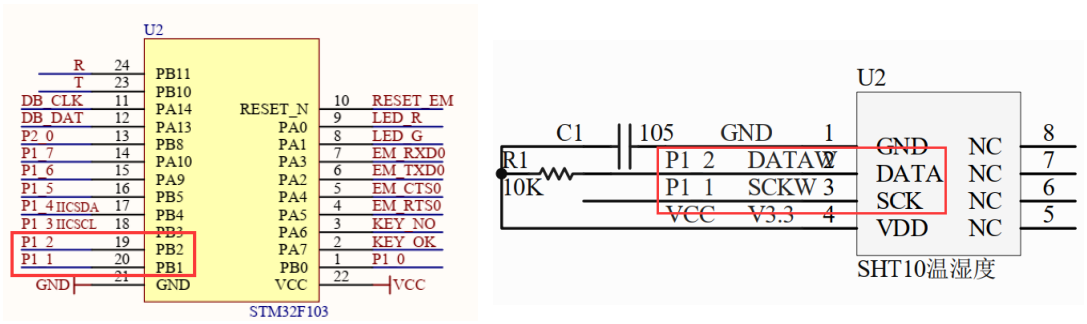


图 1-13 温湿度传感器电路原理图

温湿度传感器的时钟线连接到了 STM32 的 PB1 口，数据线连接到了 STM32 的 PB2 口。

1.1.4. 实验内容

本实验是通过 STM32 的 PB1、PB2 口模拟 SHT1x 的读取时序，读取 SHT1x 的温湿度数据，读取到温湿度之后通过串口和 OLED 屏打印出来。根据温湿度传感器 SHT1x 的工作原理以及温湿度数据读取时序，通过编程实现温湿度值的采集：

1) 打开 Manage Run-Time Environment，勾选 I2C 和 USART

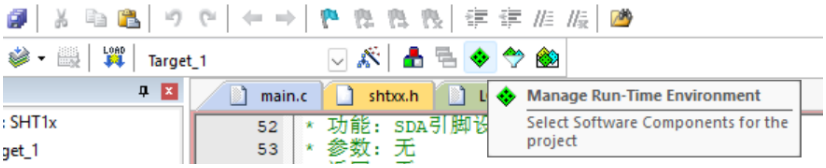


图 1-14 Manage Run-Time Environment

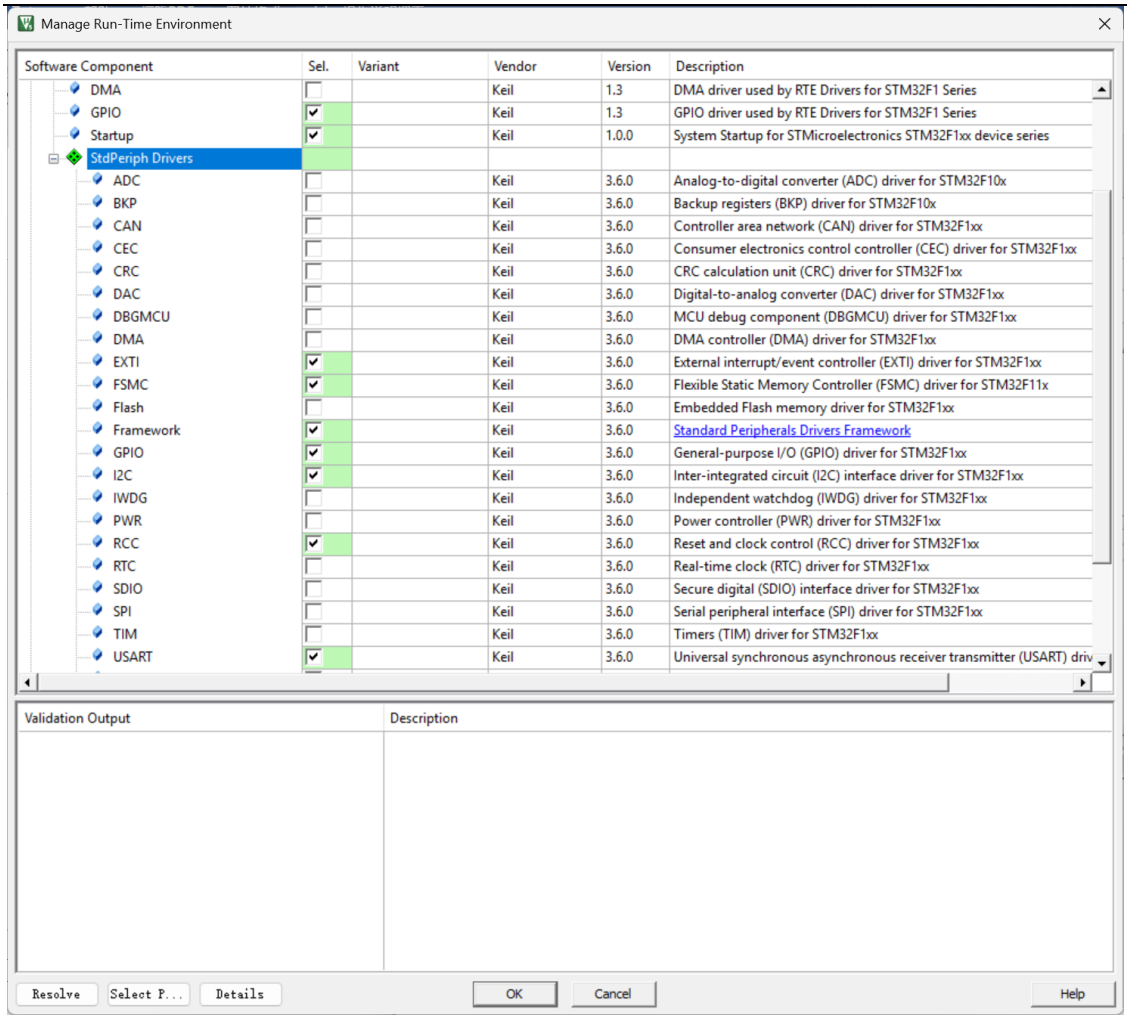


图 1-15 勾选 I2C 和 USART

2) 打开 LQ12864.c，查看 OLED_Display

```
void OLED_Display( uint8_t line, char *str )
{
   uint8_t Line_Temp;
   uint8_t Str_Length = 0, count, i;
   uint8_t Str_Error = 0, Line_Error = 0;


   switch ( line )
   {
   case OLED_Line_1:
      Line_Temp = 0;
      break;

   case OLED_Line_2:
      Line_Temp = 2;
      break;

   case OLED_Line_3:
      Line_Temp = 4;
      break;

   case OLED_Line_4:
```

```
        Line_Temp = 6;
        break;

    default:

        OLED_P8x16StrLen( 0, 1, "Input Line Error", 16 );
        Line_Error = 1;
        goto out;
        break;

    }

    Str_Length = strlen( str );

    if( Str_Length <= 16 && Str_Length > 0 )
    {
        OLED_P8x16StrLen( 0, Line_Temp, str, Str_Length );
    }

    else if( Str_Length > 16 && Str_Length <= ( 16 * 4 ) )
    {
        count = Str_Length / 16;

        for( i = 0; i < count; i++ )
        {
            OLED_P8x16StrLen( 0, Line_Temp, &str[i * 16], 16 );
            Line_Temp += 2;
        }

        OLED_P8x16StrLen( 0, Line_Temp, &str[i * 16], Str_Length % 16 );

    }

    else
    {
        OLED_P8x16StrLen( 0, 2, "Input Str Error!", 16 );
        Str_Error = 1;
        goto out;
    }

out:
    if( Str_Error == 1 )
        OLED_P8x16StrLen( 0, 4, "Check Str Length", 16 );

    if( Line_Error == 1 )
        OLED_P8x16StrLen( 0, 4, "Check Line Num!", 15 );


}
```

3) 打开 shtxx.h 文件，查看 SHT1x 时钟、数据线 IO 口的宏定义

```
#ifndef SHTXX_H
#define SHTXX_H


#define SCL_GPIO_PIN GPIO_Pin_1
#define SDA_GPIO_PIN GPIO_Pin_2
```

```
#define  SCL_GPIO_PORT        GPIOB
#define  SCL_GPIO_CLK         RCC_APB2Periph_GPIOB


#define  SDA_GPIO_PORT        GPIOB
#define  SDA_GPIO_CLK         RCC_APB2Periph_GPIOB


#define Clr_IIC_SCL()   GPIO_ResetBits(SCL_GPIO_PORT, SCL_GPIO_PIN)
#define Set_IIC_SCL()   GPIO_SetBits(SCL_GPIO_PORT, SCL_GPIO_PIN)

#define Set_IIC_SDA()   GPIO_SetBits(SDA_GPIO_PORT, SDA_GPIO_PIN)
#define Clr_IIC_SDA()   GPIO_ResetBits(SDA_GPIO_PORT, SDA_GPIO_PIN)
#define READ_SDA()      GPIO_ReadInputDataBit(SDA_GPIO_PORT, SDA_GPIO_PIN)
#define WRITE_SDA(x)    GPIO_WriteBit(SDA_GPIO_PORT, SDA_GPIO_PIN, x)


enum {TEMP,HUMI};

void IIC_GPIOConfig(void);
void SDA_OUT(void);
void SDA_IN(void);

void SHTXX_Init(void);
void FS_Init(void);
static char WriteByte(unsigned char value);
static char ReadByte(unsigned char ack);
void ConnectionReset(void);
static void TransStart(void);
unsigned char Measure(unsigned char *p_value, unsigned char *p_checkum, unsigned char mode);
void SHTXX_Cal(unsigned short *temp, unsigned short *humi, float *f_temp, float *f_humi );
int SHT11_Read_Data(char* pt, char* ph);

#endif
```

4) 打开 shtxx.c 文件，查看 SHT1x 模块 IO 初始化

```
#include "shtxx.h"
// #include "delay.h"
#include "stm32f10x.h"
#include "stm32f10x_rcc.h"
#include "stm32f10x_gpio.h"

#define noACK 0
#define ACK   1

#define STATUS_REG_W 0x06   //000   0011    0
#define STATUS_REG_R 0x07   //000   0011    1
#define MEASURE_TEMP 0x03   //000   0001    1
#define MEASURE_HUMI 0x05   //000   0010    1
void SHTXX_GPIOConfig(void)
{
  GPIO_InitTypeDef       GPIO_InitStructure;

  RCC_APB2PeriphClockCmd(SDA_GPIO_CLK | SCL_GPIO_CLK, ENABLE);
  //PB1 PB2
```

```
  GPIO_InitStructure.GPIO_Pin = SCL_GPIO_PIN | SDA_GPIO_PIN;
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
  GPIO_Init(SDA_GPIO_PORT, &GPIO_InitStructure);
}
```

5) 通过温湿度传感器测量温度和湿度

```
uint8_t Measure(unsigned char *p_value, unsigned char *p_checksum, unsigned char mode)
{
  unsigned error = 0, i = 0;

  ConnectionReset();
  switch(mode)
  {
  case TEMP:
    error = WriteByte(MEASURE_TEMP);break;
  case HUMI:
    error = WriteByte(MEASURE_HUMI);break;
  default:
    break;
  }
  if(error)  //写命令异常
  {
    return 1;
  }

  SDA_IN();
  while(i++<400000 && READ_SDA());   //测量结束或者超时

  if(READ_SDA())  // 超时
  {
    return 1;
  }
  *(p_value +1)  =ReadByte(ACK);    //读第一个字节 (MSB)
  *(p_value)=ReadByte(ACK);    //读第二个字节 (LSB)
  *p_checksum =ReadByte(noACK);  //读校验值
  return 0;
}
```

6) 将读取的温湿度的值转化为可识别的值

```
void SHTXX_Cal(uint16_t *temp, uint16_t *humi, float *f_temp, float *f_humi )
{

  const float C1=-4.0;              // for 12 Bit
  const float C2=+0.0405;           // for 12 Bit
  const float C3=-0.0000028;        // for 12 Bit
  const float T1=+0.01;             // for 14 Bit @ 5V
  const float T2=+0.00008;          // for 14 Bit @ 5V

  float rh;             // rh:      Humidity [Ticks] 12 Bit
  float t;          // t:       Temperature [Ticks] 14 Bit
  float rh_lin;                     // rh_lin:  Humidity linear
  float rh_true;                    // rh_true: Temperature compensated humidity
  float t_C;                       // t_C   :  Temperature

  rh = (float)(*humi);
```

```
  t = (float)(*temp);

  t_C=t*0.01 - 40;                  //calc. temperature from ticks
  rh_lin=C3*rh*rh + C2*rh + C1;     //calc. humidity from ticks to [%RH]
  rh_true=(t_C-25)*(T1+T2*rh)+rh_lin;   //calc. temperature compensated humidity [%RH]
  if(rh_true>100)rh_true=100;       //cut if the value is outside of
  if(rh_true<0.1)rh_true=0.1;       //the physical possible range

  *f_temp=t_C;              //return temperature
  *f_humi=rh_true;              //return humidity[%RH]

}
```

7) 打开 main.c，查看 main 函数

```
int main(void)
{
  uint8_t error, checksum;
  uint16_t humi_val, temp_val;
  char temp[16] = {0},humi[16] = {0};
  float f_humi, f_temp;

  delay_init(72);
  uart1_init();
  OLED_Init();
  SHTXX_Init();
  OLED_Display(OLED_Line_2, "HumiTemp Exp.");
  printf("Stm32 Sensors example start !\r\n");

  while(1)
  {
    error = 0;
    error += Measure((unsigned char*) &temp_val,&checksum,TEMP);  //测量温度
    error += Measure((unsigned char*) &humi_val,&checksum,HUMI);  //测量湿度

    if (error == 0)
    {
      //通过公式将读取的温湿度的值转换成可识别的值
      SHTXX_Cal(&temp_val, &humi_val, &f_temp, &f_humi);
      sprintf(temp, "Temp = %.1fC", f_temp);
      sprintf(humi, "Humi = %.1f%%", f_humi);

      OLED_Clear_Line(OLED_Line_3);
      OLED_Clear_Line(OLED_Line_4);
      OLED_Display(OLED_Line_3, temp);   //OLED 屏打印温湿度信息
      OLED_Display(OLED_Line_4, humi);

      printf("Temp = %.1f℃ , Humi = %.1f%%\r\n", f_temp, f_humi);  //串口打印温湿度信息
    }
    delay_ms(1000);
  }
}
```

接下来我们来看一下温湿度传感器实验的实验流程图：

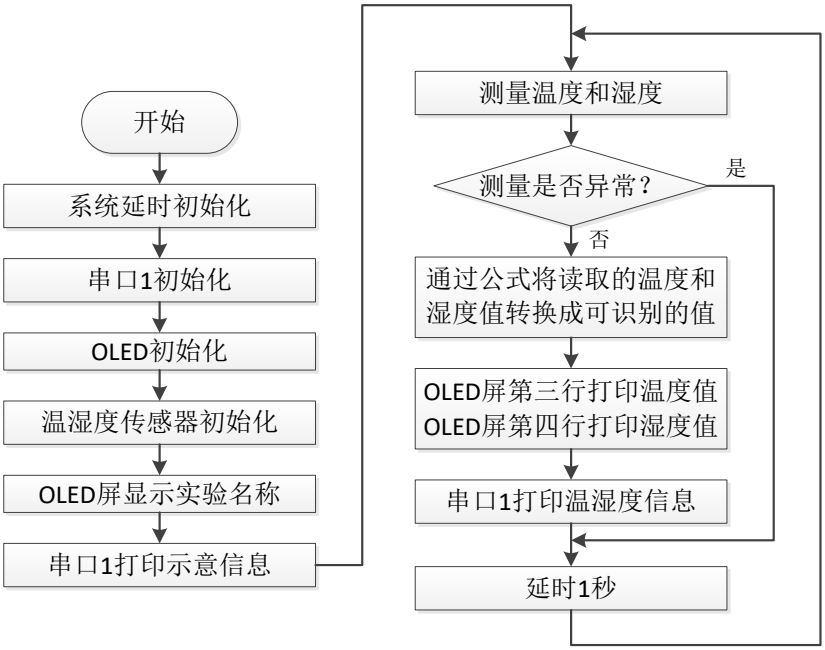


图 1-16 温湿度传感器实验流程图

1.1.5. 实验步骤

1) 将传感器正确插入 STM32 开发板传感器接口，OLED 显示模块插到 STM32 模块上，如图 1-17 所示。

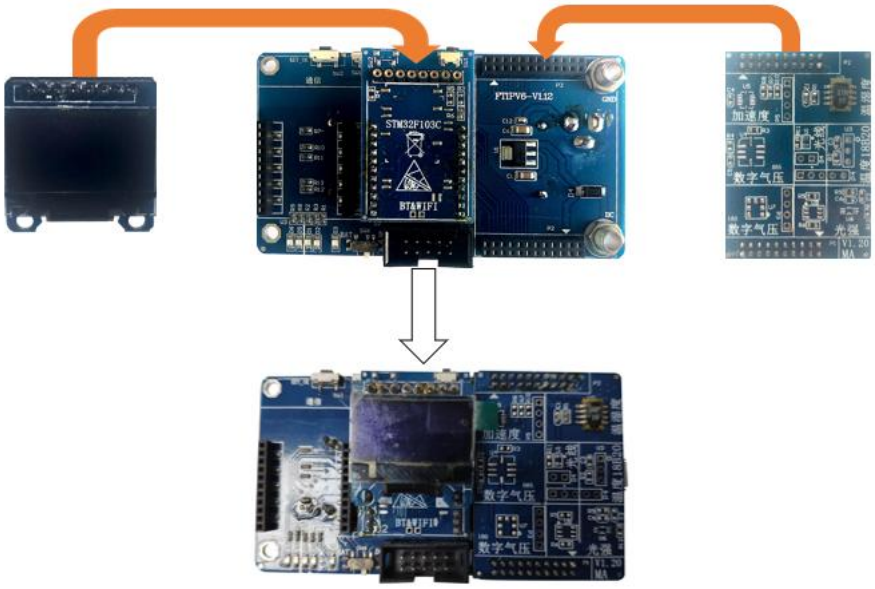
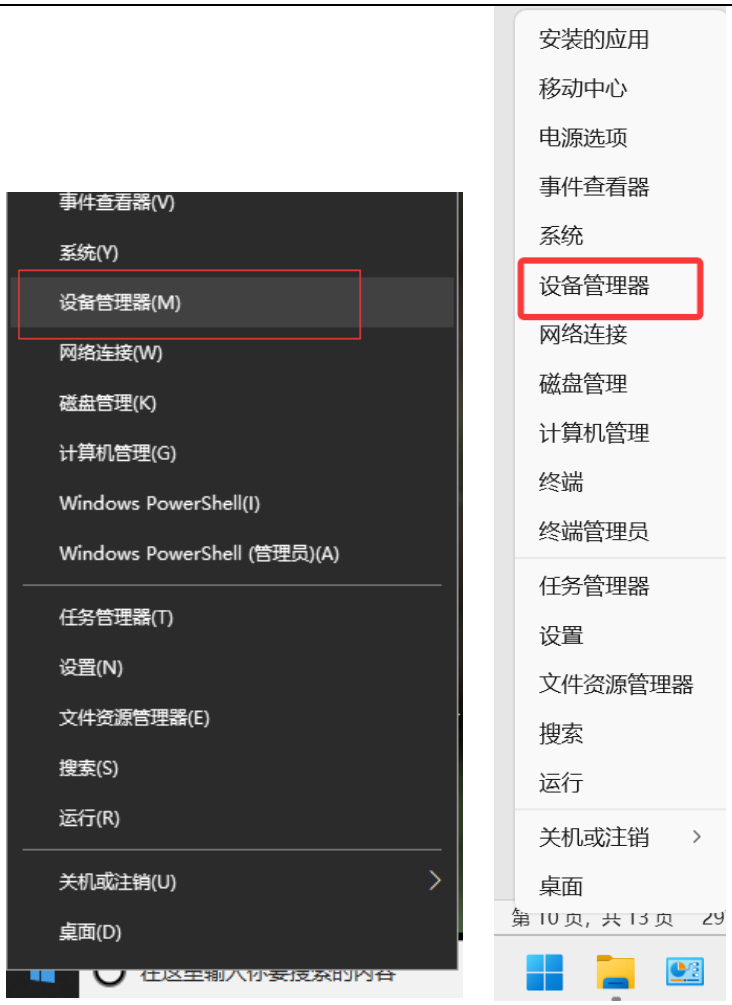


图 1-17 将传感器插入到开发板，OLED 模块插到 STM23 模块

2) 使用 USB 方口线连接 ARM J-LINK 仿真器和 PC 机，用 20 针排线连接 ARM J-LINK 仿真器和下载调试板，10 针排线连接下载调试板和 STM32 开发板，再使用串口线将下载调试板连接至 PC，使用 DC 5V 电源给开发板供电，如图 1-18 所示。

图 1-18 硬件连接图

3) 在 PC 机上查看 STM32 开发板连接到 PC 机的端口号，如果串口线连接到 PC 机的 DB9 口，通常端口号默认为 COM1，如果串口线是通过 U 转串连接到电脑 USB 口上的，则需要打开设备管理器查看正确的端口号，如



4) 图 1-19 所示，打开设备管理器（不同的 Windows 系统打开方式可能不同）。

5)

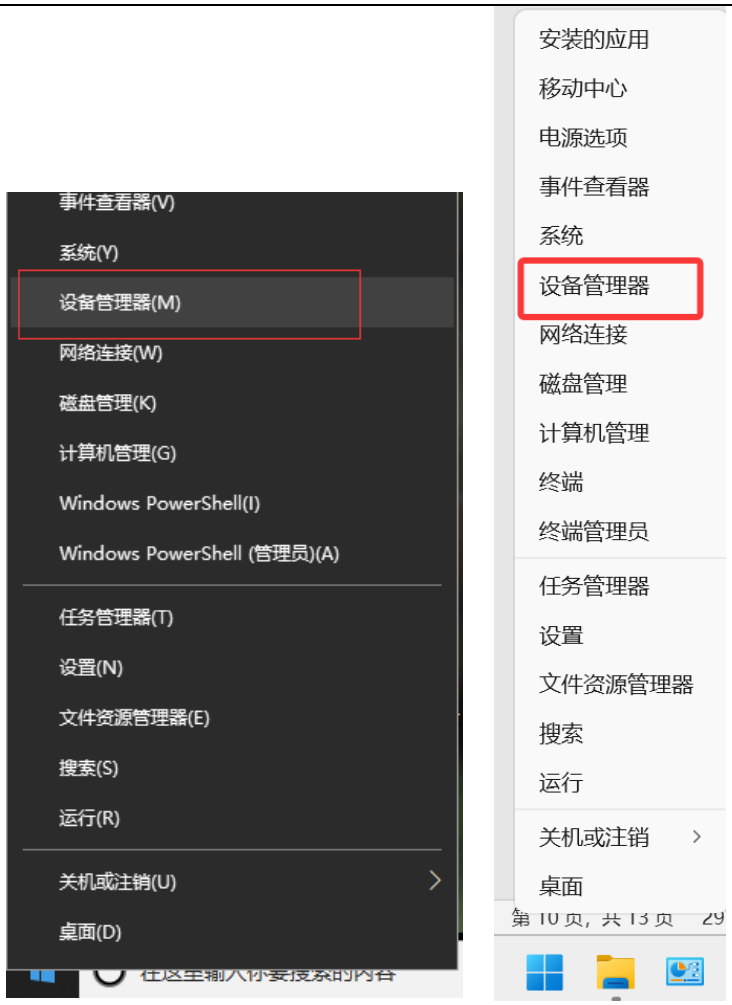


图 1-19 打开设备管理器（左 win10，右 win11）

然后在设备管理器的端口下查看串口使用的端口号，如图 1-20 所示。

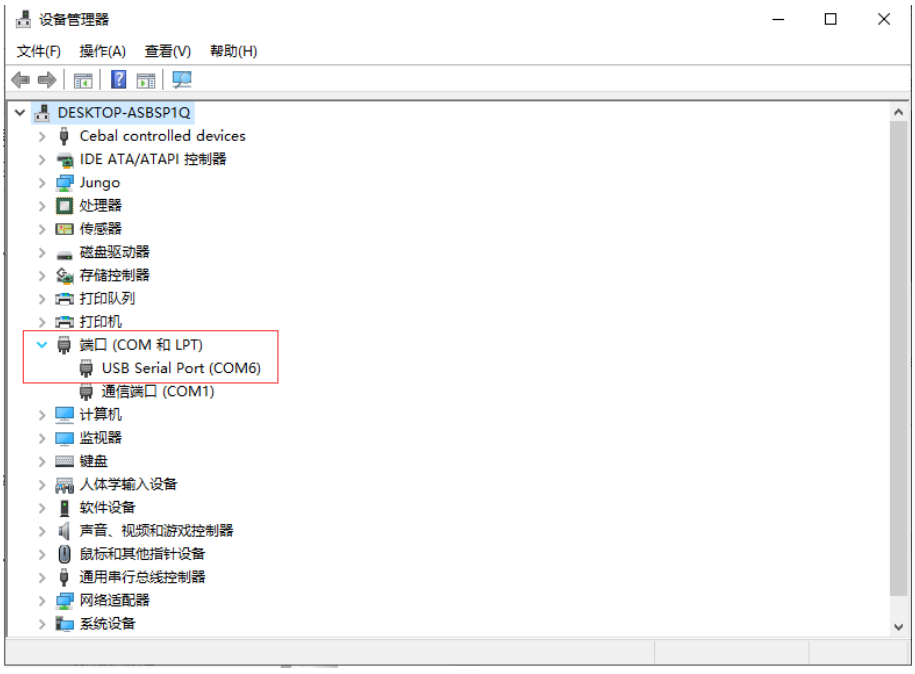


图 1-20 在设备管理器的端口下查看端口号

6) 用 Keil5 开发环境打开实验例程，点击 Rebuild All 重新编译工程，如图 1-21 所示。

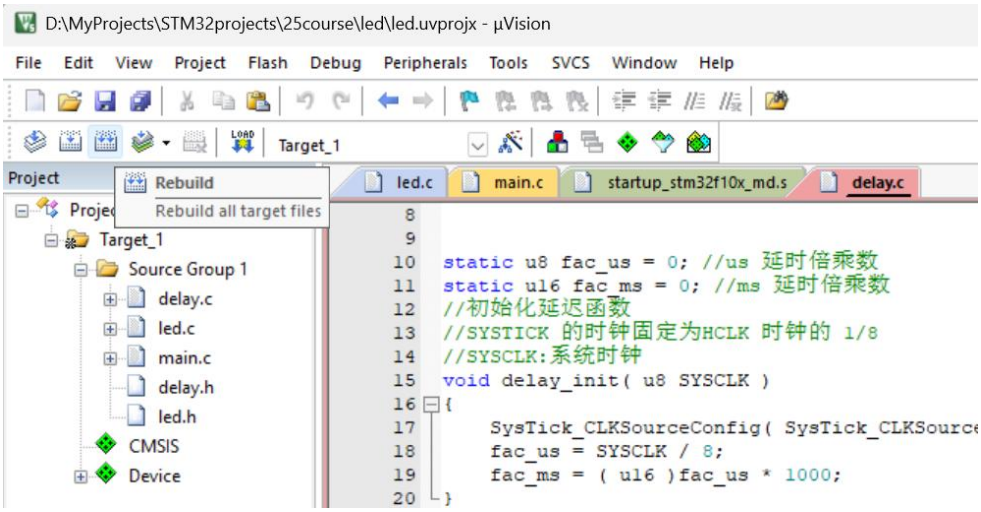


图 1-21 重新编译

7) 点击 Download 将程序下载到 STM32 开发板中，即可观察到开发板程序运行（如按照教程勾选”Reset and Run”），如图 1-22 所示。也可以将 STM32 开发板重新上电或者按下复位按钮让刚才下载的程序重新运行。

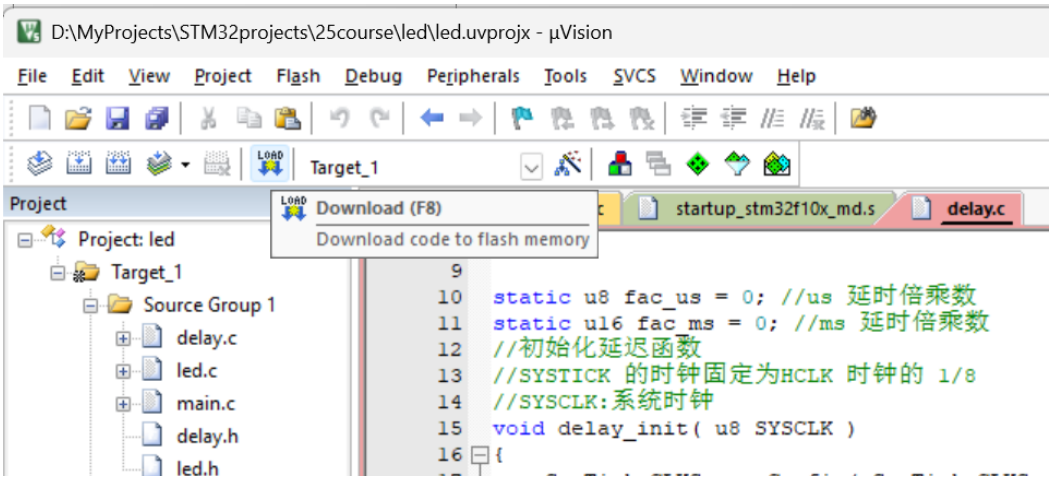


图 1-22 下载程序

8) （可选）下载完后可以点击 Debug 让程序进入调试模式，如图 1-23 所示；

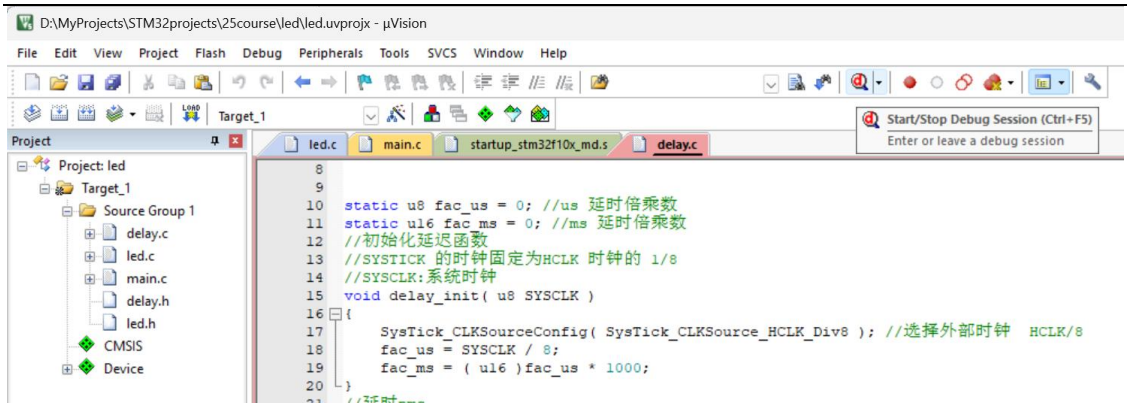


图 1-23 运行程序

9) 程序成功运行后，在 PC 机上打开串口调试小助手，设置波特率为 115200，端口号选择步骤 3 查看到的端口，不勾选自动清空，其他设置采用默认值，点击打开串口，如图 1-24 所示。

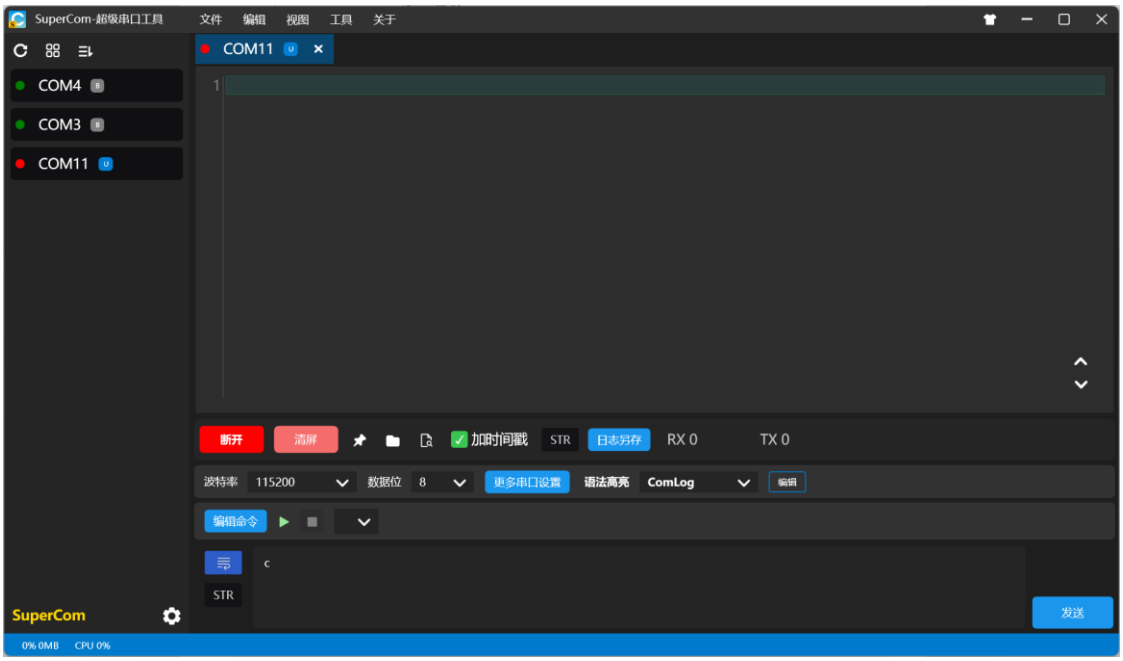


图 1-24 打开串口助手

10) 观察串口调试工具接收区和 OLED 屏的打印信息。

1.1.6. 实验现象

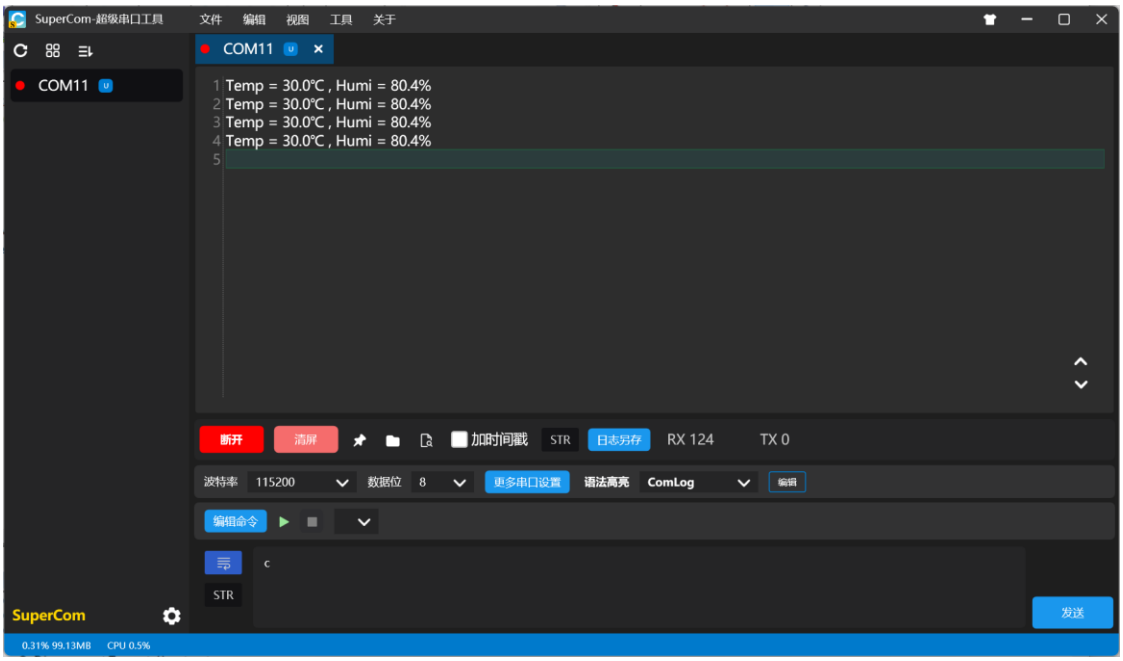


图 1-25 串口打印信息

在串口调试助手接收区看到有实时温湿度数值不断被打印，此时用手轻轻触摸传感器或者对传感器哈气，会发现温湿度的值均有所上升，如图 1-25 所示。

OLED 屏同样显示温湿度信息（OLED 屏没有显示的话可以断电再上电试一下），第二行打印实验名称，第三行打印温度值，第四行打印湿度值，如图 1-26 所示。

注：本实验 OLED 屏所起到的作用同串口是一致的，用来显示实验信息，使用串口或者 OLED 屏任意一个看到实验现象即可，后续的传感器实验同时使用到串口和 OLED 屏的也是这样。

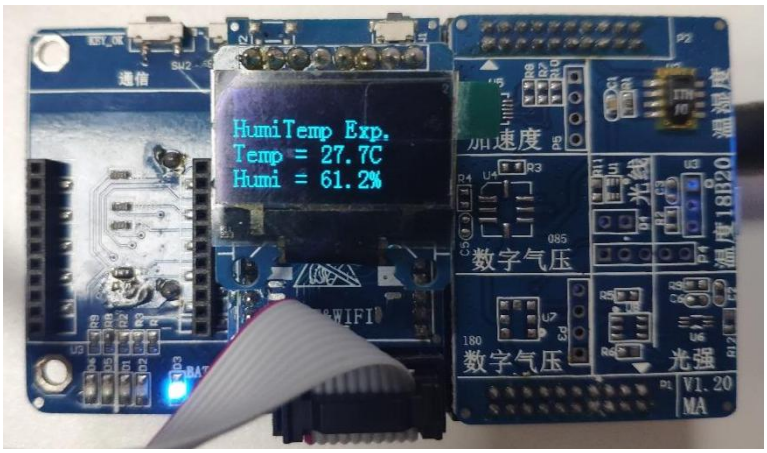


图 1-26 OLED 屏显示温湿度信息

1.1.7. 思考题

若实现温度变化超过 0.5 度打印一次。代码需要如何实现呢？

请思考并动手修改代码，实现以上功能。

2. 附录：源代码

2.1. 温湿度传感器实验

1) LQ12864.c

```
#include "stm32f10x_rcc.h"
#include "stm32f10x_gpio.h"

#include "LQ12864.h"
#include <string.h>

#define high 1
#define low 0

//´ ø² Îº ê£¬¿ÉÒÔÏñÄÚÁª ¯ÊýÒ»ÑùÊ¹ ÓÃ
#define OLED_SCL(a)    if (a)     \
                                    GPIO_SetBits(GPIOB, GPIO_Pin_6);\
                                    else                \
                                    GPIO_ResetBits(GPIOB,GPIO_Pin_6)

#define OLED_SDA(a)   if (a)     \
                                    GPIO_SetBits(GPIOB,GPIO_Pin_7);\
                                    else                \
                                    GPIO_ResetBits(GPIOB,GPIO_Pin_7)


#define NOPS  __asm volatile ("nop\n nop\n nop\n nop\n nop")

#define  Brightness         0xCF
#define X_WIDTH         128
#define Y_WIDTH         64



static const char  F8X16[] =

{
   0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x
00, // 0
   0x00, 0x00, 0x00, 0xF8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x33, 0x30, 0x00, 0x00, 0x
00, //! 1
   0x00, 0x10, 0x0C, 0x06, 0x10, 0x0C, 0x06, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x
00, //" 2
   0x40, 0xC0, 0x78, 0x40, 0xC0, 0x78, 0x40, 0x00, 0x04, 0x3F, 0x04, 0x04, 0x3F, 0x04, 0x04, 0
x00, //# 3
   0x00, 0x70, 0x88, 0xFC, 0x08, 0x30, 0x00, 0x00, 0x00, 0x18, 0x20, 0xFF, 0x21, 0x1E, 0x00, 0
x00, //$ 4
   0xF0, 0x08, 0xF0, 0x00, 0xE0, 0x18, 0x00, 0x00, 0x00, 0x21, 0x1C, 0x03, 0x1E, 0x21, 0x1E, 0
x00, //% 5
   0x00, 0xF0, 0x08, 0x88, 0x70, 0x00, 0x00, 0x00, 0x1E, 0x21, 0x23, 0x24, 0x19, 0x27, 0x21, 0x
10, //& 6
   0x10, 0x16, 0x0E, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x
00, //' 7
```

```
    0x00, 0x00, 0x00, 0xE0, 0x18, 0x04, 0x02, 0x00, 0x00, 0x00, 0x00, 0x07, 0x18, 0x20, 0x40, 0x
00, //( 8
    0x00, 0x02, 0x04, 0x18, 0xE0, 0x00, 0x00, 0x00, 0x00, 0x40, 0x20, 0x18, 0x07, 0x00, 0x00, 0x
00, //) 9
    0x40, 0x40, 0x80, 0xF0, 0x80, 0x40, 0x40, 0x00, 0x02, 0x02, 0x01, 0x0F, 0x01, 0x02, 0x02, 0x
00, //* 10
    0x00, 0x00, 0x00, 0xF0, 0x00, 0x00, 0x00, 0x00, 0x01, 0x01, 0x01, 0x1F, 0x01, 0x01, 0x01, 0x
00, //+ 11
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0xB0, 0x70, 0x00, 0x00, 0x00, 0x00, 0x
00, //, 12
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x
01, //- 13
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x00, 0x00, 0x00, 0x00, 0x
00, //. 14
    0x00, 0x00, 0x00, 0x00, 0x80, 0x60, 0x18, 0x04, 0x00, 0x60, 0x18, 0x06, 0x01, 0x00, 0x00, 0x
00, /// 15
    0x00, 0xE0, 0x10, 0x08, 0x08, 0x10, 0xE0, 0x00, 0x00, 0x0F, 0x10, 0x20, 0x20, 0x10, 0x0F, 0x
00, //0 16
    0x00, 0x10, 0x10, 0xF8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x20, 0x3F, 0x20, 0x20, 0x00, 0x
00, //1 17
    0x00, 0x70, 0x08, 0x08, 0x08, 0x88, 0x70, 0x00, 0x00, 0x30, 0x28, 0x24, 0x22, 0x21, 0x30, 0x
00, //2 18
    0x00, 0x30, 0x08, 0x88, 0x88, 0x48, 0x30, 0x00, 0x00, 0x18, 0x20, 0x20, 0x20, 0x11, 0x0E, 0x
00, //3 19
    0x00, 0x00, 0xC0, 0x20, 0x10, 0xF8, 0x00, 0x00, 0x00, 0x07, 0x04, 0x24, 0x24, 0x3F, 0x24, 0x
00, //4 20
    0x00, 0xF8, 0x08, 0x88, 0x88, 0x08, 0x08, 0x00, 0x00, 0x19, 0x21, 0x20, 0x20, 0x11, 0x0E, 0x
00, //5 21
    0x00, 0xE0, 0x10, 0x88, 0x88, 0x18, 0x00, 0x00, 0x00, 0x0F, 0x11, 0x20, 0x20, 0x11, 0x0E, 0x
00, //6 22
    0x00, 0x38, 0x08, 0x08, 0xC8, 0x38, 0x08, 0x00, 0x00, 0x00, 0x00, 0x3F, 0x00, 0x00, 0x00, 0x
00, //7 23
    0x00, 0x70, 0x88, 0x08, 0x08, 0x88, 0x70, 0x00, 0x00, 0x1C, 0x22, 0x21, 0x21, 0x22, 0x1C, 0x
00, //8 24
    0x00, 0xE0, 0x10, 0x08, 0x08, 0x10, 0xE0, 0x00, 0x00, 0x00, 0x31, 0x22, 0x22, 0x11, 0x0F, 0x
00, //9 25
    0x00, 0x00, 0x00, 0xC0, 0xC0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x30, 0x00, 0x00, 0x
00, //: 26
    0x00, 0x00, 0x00, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x60, 0x00, 0x00, 0x00, 0x
00, //; 27
    0x00, 0x00, 0x80, 0x40, 0x20, 0x10, 0x08, 0x00, 0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x
00, //< 28
    0x40, 0x40, 0x40, 0x40, 0x40, 0x40, 0x40, 0x00, 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x
00, //= 29
    0x00, 0x08, 0x10, 0x20, 0x40, 0x80, 0x00, 0x00, 0x00, 0x20, 0x10, 0x08, 0x04, 0x02, 0x01, 0x
00, //> 30
    0x00, 0x70, 0x48, 0x08, 0x08, 0x08, 0xF0, 0x00, 0x00, 0x00, 0x00, 0x30, 0x36, 0x01, 0x00, 0x
00, //? 31
    0xC0, 0x30, 0xC8, 0x28, 0xE8, 0x10, 0xE0, 0x00, 0x07, 0x18, 0x27, 0x24, 0x23, 0x14, 0x0B, 0
x00, //@ 32
    0x00, 0x00, 0xC0, 0x38, 0xE0, 0x00, 0x00, 0x00, 0x20, 0x3C, 0x23, 0x02, 0x02, 0x27, 0x38, 0
x20, //A 33
    0x08, 0xF8, 0x88, 0x88, 0x88, 0x70, 0x00, 0x00, 0x20, 0x3F, 0x20, 0x20, 0x20, 0x11, 0x0E, 0x
00, //B 34
    0xC0, 0x30, 0x08, 0x08, 0x08, 0x08, 0x38, 0x00, 0x07, 0x18, 0x20, 0x20, 0x20, 0x10, 0x08, 0x
00, //C 35
```

```
0x08, 0xF8, 0x08, 0x08, 0x08, 0x10, 0xE0, 0x00, 0x20, 0x3F, 0x20, 0x20, 0x20, 0x10, 0x0F, 0x
00, //D 36
0x08, 0xF8, 0x88, 0x88, 0xE8, 0x08, 0x10, 0x00, 0x20, 0x3F, 0x20, 0x20, 0x23, 0x20, 0x18, 0x
00, //E 37
0x08, 0xF8, 0x88, 0x88, 0xE8, 0x08, 0x10, 0x00, 0x20, 0x3F, 0x20, 0x00, 0x03, 0x00, 0x00, 0x
00, //F 38
0xC0, 0x30, 0x08, 0x08, 0x08, 0x38, 0x00, 0x00, 0x07, 0x18, 0x20, 0x20, 0x22, 0x1E, 0x02, 0x
00, //G 39
0x08, 0xF8, 0x08, 0x00, 0x00, 0x08, 0xF8, 0x08, 0x20, 0x3F, 0x21, 0x01, 0x01, 0x21, 0x3F, 0x
20, //H 40
0x00, 0x08, 0x08, 0xF8, 0x08, 0x08, 0x00, 0x00, 0x00, 0x20, 0x20, 0x3F, 0x20, 0x20, 0x00, 0x
00, //I 41
0x00, 0x00, 0x08, 0x08, 0xF8, 0x08, 0x08, 0x00, 0xC0, 0x80, 0x80, 0x80, 0x7F, 0x00, 0x00, 0x
00, //J 42
0x08, 0xF8, 0x88, 0xC0, 0x28, 0x18, 0x08, 0x00, 0x20, 0x3F, 0x20, 0x01, 0x26, 0x38, 0x20, 0x
00, //K 43
0x08, 0xF8, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x3F, 0x20, 0x20, 0x20, 0x20, 0x30, 0x
00, //L 44
0x08, 0xF8, 0xF8, 0x00, 0xF8, 0xF8, 0x08, 0x00, 0x20, 0x3F, 0x00, 0x3F, 0x00, 0x3F, 0x20, 0
x00, //M 45
0x08, 0xF8, 0x30, 0xC0, 0x00, 0x08, 0xF8, 0x08, 0x20, 0x3F, 0x20, 0x00, 0x07, 0x18, 0x3F, 0
x00, //N 46
0xE0, 0x10, 0x08, 0x08, 0x08, 0x10, 0xE0, 0x00, 0x0F, 0x10, 0x20, 0x20, 0x20, 0x10, 0x0F, 0x
00, //O 47
0x08, 0xF8, 0x08, 0x08, 0x08, 0x08, 0xF0, 0x00, 0x20, 0x3F, 0x21, 0x01, 0x01, 0x01, 0x00, 0x
00, //P 48
0xE0, 0x10, 0x08, 0x08, 0x08, 0x10, 0xE0, 0x00, 0x0F, 0x18, 0x24, 0x24, 0x38, 0x50, 0x4F, 0x
00, //Q 49
0x08, 0xF8, 0x88, 0x88, 0x88, 0x88, 0x70, 0x00, 0x20, 0x3F, 0x20, 0x00, 0x03, 0x0C, 0x30, 0x
20, //R 50
0x00, 0x70, 0x88, 0x08, 0x08, 0x08, 0x38, 0x00, 0x00, 0x38, 0x20, 0x21, 0x21, 0x22, 0x1C, 0x
00, //S 51
0x18, 0x08, 0x08, 0xF8, 0x08, 0x08, 0x18, 0x00, 0x00, 0x00, 0x20, 0x3F, 0x20, 0x00, 0x00, 0x
00, //T 52
0x08, 0xF8, 0x08, 0x00, 0x00, 0x08, 0xF8, 0x08, 0x00, 0x1F, 0x20, 0x20, 0x20, 0x20, 0x1F, 0x
00, //U 53
0x08, 0x78, 0x88, 0x00, 0x00, 0xC8, 0x38, 0x08, 0x00, 0x00, 0x07, 0x38, 0x0E, 0x01, 0x00, 0x
00, //V 54
0xF8, 0x08, 0x00, 0xF8, 0x00, 0x08, 0xF8, 0x00, 0x03, 0x3C, 0x07, 0x00, 0x07, 0x3C, 0x03, 0
x00, //W 55
0x08, 0x18, 0x68, 0x80, 0x80, 0x68, 0x18, 0x08, 0x20, 0x30, 0x2C, 0x03, 0x03, 0x2C, 0x30, 0x
20, //X 56
0x08, 0x38, 0xC8, 0x00, 0xC8, 0x38, 0x08, 0x00, 0x00, 0x00, 0x20, 0x3F, 0x20, 0x00, 0x00, 0
x00, //Y 57
0x10, 0x08, 0x08, 0x08, 0xC8, 0x38, 0x08, 0x00, 0x20, 0x38, 0x26, 0x21, 0x20, 0x20, 0x18, 0x
00, //Z 58
0x00, 0x00, 0x00, 0xFE, 0x02, 0x02, 0x02, 0x00, 0x00, 0x00, 0x00, 0x7F, 0x40, 0x40, 0x40, 0x
00, //[ 59
0x00, 0x0C, 0x30, 0xC0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x06, 0x38, 0xC0, 0
x00, //\ 60
0x00, 0x02, 0x02, 0x02, 0xFE, 0x00, 0x00, 0x00, 0x00, 0x40, 0x40, 0x40, 0x7F, 0x00, 0x00, 0x
00, //] 61
0x00, 0x00, 0x04, 0x02, 0x02, 0x02, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x
00, //^ 62
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x
80, //_ 63
```

```
    0x00, 0x02, 0x02, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x
00, //` 64
    0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x19, 0x24, 0x22, 0x22, 0x22, 0x3F, 0x
20, //a 65
    0x08, 0xF8, 0x00, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x3F, 0x11, 0x20, 0x20, 0x11, 0x0E, 0x
00, //b 66
    0x00, 0x00, 0x00, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x0E, 0x11, 0x20, 0x20, 0x20, 0x11, 0x
00, //c 67
    0x00, 0x00, 0x00, 0x80, 0x80, 0x88, 0xF8, 0x00, 0x00, 0x0E, 0x11, 0x20, 0x20, 0x10, 0x3F, 0x
20, //d 68
    0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x1F, 0x22, 0x22, 0x22, 0x22, 0x13, 0x
00, //e 69
    0x00, 0x80, 0x80, 0xF0, 0x88, 0x88, 0x88, 0x18, 0x00, 0x20, 0x20, 0x3F, 0x20, 0x20, 0x00, 0x
00, //f 70
    0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x6B, 0x94, 0x94, 0x94, 0x93, 0x60, 0x
00, //g 71
    0x08, 0xF8, 0x00, 0x80, 0x80, 0x80, 0x00, 0x00, 0x20, 0x3F, 0x21, 0x00, 0x00, 0x20, 0x3F, 0x
20, //h 72
    0x00, 0x80, 0x98, 0x98, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x20, 0x3F, 0x20, 0x20, 0x00, 0x
00, //i 73
    0x00, 0x00, 0x00, 0x80, 0x98, 0x98, 0x00, 0x00, 0x00, 0xC0, 0x80, 0x80, 0x80, 0x7F, 0x00, 0x
00, //j 74
    0x08, 0xF8, 0x00, 0x00, 0x80, 0x80, 0x80, 0x00, 0x20, 0x3F, 0x24, 0x02, 0x2D, 0x30, 0x20, 0x
00, //k 75
    0x00, 0x08, 0x08, 0xF8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x20, 0x3F, 0x20, 0x20, 0x00, 0x
00, //l 76
    0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x00, 0x20, 0x3F, 0x20, 0x00, 0x3F, 0x20, 0x00, 0x
3F, //m 77
    0x80, 0x80, 0x00, 0x80, 0x80, 0x80, 0x00, 0x00, 0x20, 0x3F, 0x21, 0x00, 0x00, 0x20, 0x3F, 0x
20, //n 78
    0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00, 0x1F, 0x20, 0x20, 0x20, 0x20, 0x1F, 0x
00, //o 79
    0x80, 0x80, 0x00, 0x80, 0x80, 0x00, 0x00, 0x00, 0x80, 0xFF, 0xA1, 0x20, 0x20, 0x11, 0x0E, 0
x00, //p 80
    0x00, 0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x0E, 0x11, 0x20, 0x20, 0xA0, 0xFF, 0
x80, //q 81
    0x80, 0x80, 0x80, 0x00, 0x80, 0x80, 0x80, 0x00, 0x20, 0x20, 0x3F, 0x21, 0x20, 0x00, 0x01, 0x
00, //r 82
    0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x33, 0x24, 0x24, 0x24, 0x24, 0x19, 0x
00, //s 83
    0x00, 0x80, 0x80, 0xE0, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1F, 0x20, 0x20, 0x00, 0x
00, //t 84
    0x80, 0x80, 0x00, 0x00, 0x00, 0x80, 0x80, 0x00, 0x00, 0x1F, 0x20, 0x20, 0x20, 0x10, 0x3F, 0x
20, //u 85
    0x80, 0x80, 0x80, 0x00, 0x00, 0x80, 0x80, 0x80, 0x00, 0x01, 0x0E, 0x30, 0x08, 0x06, 0x01, 0x
00, //v 86
    0x80, 0x80, 0x00, 0x80, 0x00, 0x80, 0x80, 0x80, 0x0F, 0x30, 0x0C, 0x03, 0x0C, 0x30, 0x0F, 0
x00, //w 87
    0x00, 0x80, 0x80, 0x00, 0x80, 0x80, 0x80, 0x00, 0x00, 0x20, 0x31, 0x2E, 0x0E, 0x31, 0x20, 0x
00, //x 88
    0x80, 0x80, 0x80, 0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x81, 0x8E, 0x70, 0x18, 0x06, 0x01, 0x
00, //y 89
    0x00, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x21, 0x30, 0x2C, 0x22, 0x21, 0x30, 0x
00, //z 90
    0x00, 0x00, 0x00, 0x00, 0x80, 0x7C, 0x02, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3F, 0x40, 0x
40, //{ 91
```

```
   0x00, 0x00, 0x00, 0x00, 0xFF, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xFF, 0x00, 0x00, 0x
00, //| 92
   0x00, 0x02, 0x02, 0x7C, 0x80, 0x00, 0x00, 0x00, 0x00, 0x40, 0x40, 0x3F, 0x00, 0x00, 0x00, 0x
00, //} 93
   0x00, 0x06, 0x01, 0x01, 0x02, 0x02, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x
00, //~ 94
};

/**********************OLEDÇý¶¯³ÌÐòÓÃµÄÑÓÊ±³ÌÐò*****************************
*********/
void OLED_delay( unsigned int z )
{
   unsigned int x;
   for( x = z; x > 0; x-- )
      NOPS;
}

/************************************************
//IIC Start
************************************************/
void OLED_IIC_Start( void )
{
   OLED_SCL( high );
   OLED_SDA ( high );
   NOPS;
   OLED_SDA ( low );
   OLED_SCL ( low );
   NOPS;
}

/************************************************
//IIC Stop
************************************************/
void OLED_IIC_Stop( void )
{
   OLED_SCL ( low );
   OLED_SDA ( low );
   NOPS;
   OLED_SCL( high );
   OLED_SDA ( high );
   NOPS;
}

/************************************************
// IIC Write byte
************************************************/
void Write_IIC_Byte( unsigned char IIC_Byte )
{
   unsigned char i;
   for( i = 0; i < 8; i++ )
   {
      if( IIC_Byte & 0x80 )
         OLED_SDA( high );
      else
         OLED_SDA( low );
      OLED_SCL( high );
      NOPS;
```

```
    OLED_SCL( low );
    IIC_Byte <<= 1;
    NOPS;
  }
  OLED_SDA( high );
  OLED_SCL( high );
  NOPS;
  OLED_SCL( low );
  NOPS;
}

/**********************OLEDÐ´ Êý¾Ý************************************/
void OLED_WrDat( unsigned char IIC_Data )
{
  OLED_IIC_Start();
  Write_IIC_Byte( 0x78 );
  Write_IIC_Byte( 0x40 );                    //write data
  Write_IIC_Byte( IIC_Data );
  OLED_IIC_Stop();
}

/**********************OLEDÐ´ ÃüÁî************************************/
void OLED_WrCmd( unsigned char IIC_Command )
{
  OLED_IIC_Start();
  Write_IIC_Byte( 0x78 );          //Slave address,SA0=0
  Write_IIC_Byte( 0x00 );                    //write command
  Write_IIC_Byte( IIC_Command );
  OLED_IIC_Stop();
}

/**********************OLED ÉèÖÃ×ø±ê************************************/
void OLED_Set_Pos( unsigned char x, unsigned char y )
{
  OLED_WrCmd( 0xb0 + y );
  OLED_WrCmd( ( ( x & 0xf0 ) >> 4 ) | 0x10 );
  OLED_WrCmd( ( x & 0x0f ) );
}

/**********************OLEDÈ«ÆÁ************************************/
void OLED_Fill( unsigned char bmp_dat )
{
  unsigned char y, x;
  for( y = 0; y < 8; y++ )
  {
    OLED_WrCmd( 0xb0 + y );
    OLED_WrCmd( 0x01 );
    OLED_WrCmd( 0x10 );
    for( x = 0; x < X_WIDTH; x++ )
      OLED_WrDat( bmp_dat );
  }
}

////////////////////////////////////////////
//////OLED¶ùÐÐÇåÆÁº¯Êý ²ÎÊýÎª ÐèÒª ÇåÆÁµÄÐÐº Å///
/////ÐÐº ÅÓÃ¶þ½øÖÆµÄ • ½Ê½±íÊ¾
/*     eg:0x01   µÚ1ÐÐ   0x02    µÚ2ÐÐ   0x04   µÚ3ÐÐ    0x08  µÚ4ÐÐ
```

```
0X03    µÚ1ÐÐº ÍµÚ¶þÐÐ
0X07    µÚ1¡ ¢2¡ ¢3ÐÐ
*/
////////////////////////////////////////
void OLED_Clear_Line( uint8_t LineNum )
{
   char temp[16] = {"               "};

   if( ( LineNum & OLED_Line_1 ) == OLED_Line_1 )
      OLED_P8x16StrLen( 0, 0, temp, 16 );

   if( ( LineNum & OLED_Line_2 ) == OLED_Line_2 )
      OLED_P8x16StrLen( 0, 2, temp, 16 );

   if( ( LineNum & OLED_Line_3 ) == OLED_Line_3 )
      OLED_P8x16StrLen( 0, 4, temp, 16 );

   if( ( LineNum & OLED_Line_4 ) == OLED_Line_4 )
      OLED_P8x16StrLen( 0, 6, temp, 16 );

}


/**********************OLED¸´Î»************************************/
void OLED_CLS( void )
{
   unsigned char y, x;
   for( y = 0; y < 8; y++ )
   {
      OLED_WrCmd( 0xb0 + y );
      OLED_WrCmd( 0x01 );
      OLED_WrCmd( 0x10 );
      for( x = 0; x < X_WIDTH; x++ )
         OLED_WrDat( 0 );
   }
}

/**********************OLED³ õÊ¼»¯************************************/
void OLED_Init( void )
{
   IIC_GPIO_Config();
   OLED_delay( 500 ); //³ õÊ¼»¯Ö®Ç°  µÄÑÓÊ±º ÜÖØÒª £¡
   OLED_WrCmd( 0xae ); //--turn off oled panel
   OLED_WrCmd( 0x00 ); //---set low column address
   OLED_WrCmd( 0x10 ); //---set high column address
   OLED_WrCmd( 0x40 ); //--set start line address  Set Mapping RAM Display Start Line (0x00~0
x3F)
   OLED_WrCmd( 0x81 ); //--set contrast control register
   OLED_WrCmd( Brightness ); // Set SEG Output Current Brightness
   OLED_WrCmd( 0xa1 ); //--Set SEG/Column Mapping     0xa0×óÓÒ • ´ÖÃ 0xa1Õý³ £
   OLED_WrCmd( 0xc8 ); //Set COM/Row Scan Direction   0xc0ÉÏÏÂ • ´ÖÃ 0xc8Õý³ £
   OLED_WrCmd( 0xa6 ); //--set normal display
   OLED_WrCmd( 0xa8 ); //--set multiplex ratio(1 to 64)
   OLED_WrCmd( 0x3f ); //--1/64 duty
   OLED_WrCmd( 0xd3 ); //-set display offset       Shift Mapping RAM Counter (0x00~0x3F)
   OLED_WrCmd( 0x00 ); //-not offset
   OLED_WrCmd( 0xd5 ); //--set display clock divide ratio/oscillator frequency
```

```
   OLED_WrCmd( 0x80 ); //--set divide ratio, Set Clock as 100 Frames/Sec
   OLED_WrCmd( 0xd9 ); //--set pre-charge period
   OLED_WrCmd( 0xf1 ); //Set Pre-Charge as 15 Clocks & Discharge as 1 Clock
   OLED_WrCmd( 0xda ); //--set com pins hardware configuration
   OLED_WrCmd( 0x12 );
   OLED_WrCmd( 0xdb ); //--set vcomh
   OLED_WrCmd( 0x40 ); //Set VCOM Deselect Level
   OLED_WrCmd( 0x20 ); //-Set Page Addressing Mode (0x00/0x01/0x02)
   OLED_WrCmd( 0x02 ); //
   OLED_WrCmd( 0x8d ); //--set Charge Pump enable/disable
   OLED_WrCmd( 0x14 ); //--set(0x10) disable
   OLED_WrCmd( 0xa4 ); // Disable Entire Display On (0xa4/0xa5)
   OLED_WrCmd( 0xa6 ); // Disable Inverse Display On (0xa6/a7)
   OLED_WrCmd( 0xaf ); //--turn on oled panel
   OLED_Fill( 0x00 ); //³ õÊ¼ÇåÆÁ
   OLED_Set_Pos( 0, 0 );
}

/*********************¹ ¦ÄÜÃèÊö£º ÏÔÊ¾8*16Ò»×é±ê×¼ASCII×Ö·û´®          ÏÔÊ¾µÄ×ø
±ê£ ¨ x,y£©£¬yÎª Ò³ ·¶Î § 0¡ «7*****************/
void OLED_P8x16Str( unsigned char x, unsigned char y, char ch[] )
{
   unsigned char c = 0, i = 0, j = 0;
   while ( ch[j] != '\0' )
   {
      c = ch[j] - 32;
      if( x > 120 && y < 5 )
      {
         x = 0;
         y += 2;
      }
      OLED_Set_Pos( x, y );
      for( i = 0; i < 8; i++ )
         OLED_WrDat( F8X16[c * 16 + i] );
      OLED_Set_Pos( x, y + 1 );
      for( i = 0; i < 8; i++ )
         OLED_WrDat( F8X16[c * 16 + i + 8] );
      x += 8;
      j++;
   }
}

/*****************¹ ¦ÄÜÃèÊö£º ÏÔÊ¾Ö¸ ¶ ¨ ³ ¤¶ÈµÄ±ê×¼ASCII×Ö·û´®£¬ö×Ö·ûËùÓ
ÃÏñËØµãÎª 8*16£¬lenÎª ×Ö·û´®³ ¤¶È*************/
void OLED_P8x16StrLen( unsigned char x, unsigned char y, char ch[], unsigned char len )
{
   unsigned char c = 0, i = 0, j = 0;
   while ( len )
   {
      c = ch[j] - 32;
      if( x > 120 && y < 5 )
      {
         x = 0;
         y += 2;
      }
      OLED_Set_Pos( x, y );
      for( i = 0; i < 8; i++ )
```

```
        OLED_WrDat( F8X16[c * 16 + i] );
    OLED_Set_Pos( x, y + 1 );
    for( i = 0; i < 8; i++ )
        OLED_WrDat( F8X16[c * 16 + i + 8] );
    x += 8;
    j++;
    len--;
  }
}

//////////////////////////////////////
//////OLEDÏÔÊ¾º¯Êý////////////
/*
ÏÔÊ¾µÄÊ±º òº£¬Èç¹ û×Ö·û´®³ ¤¶È³ ¬¹ ý16Î» ×Ô¶¯»»ÐÐ£¬×î¶à64Î»
line£º ÐÐº Å
str: ÐèÒª´ òÓ¡ µÄ×Ö·û´®(×¢Òâ£º str ×î¶à»ÄÜ³ ¬¹ ý64Î»)
*/

////////////////////////////////

void OLED_Display( uint8_t line, char *str )
{
  uint8_t Line_Temp;
  uint8_t Str_Length = 0, count, i;
  uint8_t Str_Error = 0, Line_Error = 0;


  switch ( line )
  {
  case OLED_Line_1:
    Line_Temp = 0;
    break;

  case OLED_Line_2:
    Line_Temp = 2;
    break;

  case OLED_Line_3:
    Line_Temp = 4;
    break;

  case OLED_Line_4:
    Line_Temp = 6;
    break;

  default:

    OLED_P8x16StrLen( 0, 1, "Input Line Error", 16 );
    Line_Error = 1;
    goto out;
    break;

  }

  Str_Length = strlen( str );

  if( Str_Length <= 16 && Str_Length > 0 )
```

```
    {
        OLED_P8x16StrLen( 0, Line_Temp, str, Str_Length );
    }

    else if( Str_Length > 16 && Str_Length <= ( 16 * 4 ) )
    {
        count = Str_Length / 16;

        for( i = 0; i < count; i++ )
        {
            OLED_P8x16StrLen( 0, Line_Temp, &str[i * 16], 16 );
            Line_Temp += 2;
        }

        OLED_P8x16StrLen( 0, Line_Temp, &str[i * 16], Str_Length % 16 );

    }

    else
    {
        OLED_P8x16StrLen( 0, 2, "Input Str Error!", 16 );
        Str_Error = 1;
        goto out;
    }

out:
    if( Str_Error == 1 )
        OLED_P8x16StrLen( 0, 4, "Check Str Length", 16 );

    if( Line_Error == 1 )
        OLED_P8x16StrLen( 0, 4, "Check Line Num!", 15 );


}

void IIC_GPIO_Config( void )
{
    /*¶¨ÒåÒ»¸öGPIO_InitTypeDefÀàÐÍµÄ½á¹¹Ìå*/
    GPIO_InitTypeDef GPIO_InitStructure;

    /*¿ªÆôGPIOCµÄÍâÉèÊ±ÖÓ*/
    RCC_APB2PeriphClockCmd( RCC_APB2Periph_GPIOB, ENABLE );

    /*Ñ¡ÔñÒª¿ØÖÆµÄGPIOCÒý½Å*/
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;

    /*ÉèÖÃÒý½ÅÄ£Ê½Îª ¨ ÓÃÍÆÍì Êä³ ö*/
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;

    /*ÉèÖÃÒý½ËÙÂÊÎª 50MHz */
    GPIO_InitStructure.GPIO_Speed  = GPIO_Speed_50MHz;

    /*µ÷ÓÃ¿âº¯ Êý£¬³ õÊ¼»¯GPIOC*/
    GPIO_Init( GPIOB, &GPIO_InitStructure );

    /* ¹Ø±ÕÙËùÓÐledµÆ*/
    GPIO_SetBits( GPIOB, GPIO_Pin_6 | GPIO_Pin_7 );
```

```
}

void OLED_itoa( unsigned char x, unsigned char y, unsigned char *puls )
{
   unsigned char i = 0, j = 0;
   unsigned char tan[2];


   tan[0] = *puls / 16;
   tan[1] = *puls & 0x0f;

   if( tan[0] <= 0x09 )
   {
      tan[0] = tan[0] + 16;
   }
   else
   {
      tan[0] = tan[0] + 23;
   }

   if( tan[1] <= 0x09 )
   {
      tan[1] = tan[1] + 16;
   }
   else
   {
      tan[1] = tan[1] + 23;
   }

   if( x > 120 )
   {
      x = 0;
      y++;
   }

   OLED_Set_Pos( x, y );
   for( i = 0; i < 8; i++ )
      OLED_WrDat( F8X16[tan[0] * 16 + i] );
   OLED_Set_Pos( x, y + 1 );
   for( i = 0; i < 8; i++ )
      OLED_WrDat( F8X16[tan[0] * 16 + i + 8] );
   x += 8;

   OLED_Set_Pos( x, y );
   for( i = 0; i < 8; i++ )
      OLED_WrDat( F8X16[tan[1] * 16 + i] );
   OLED_Set_Pos( x, y + 1 );
   for( i = 0; i < 8; i++ )
      OLED_WrDat( F8X16[tan[1] * 16 + i + 8] );
   x += 8;
   j++;

}
```

2) LQ12864.h

```
#ifndef LQ12864_H_
#define LQ12864_H_
```

```
#define OLED_Line_1     0x01
#define OLED_Line_2     0x02
#define OLED_Line_3     0x04
#define OLED_Line_4     0x08

void OLED_IIC_Start(void);
void OLED_IIC_Stop(void);
void Write_IIC_Byte(unsigned char IIC_Byte);
void OLED_WrDat(unsigned char IIC_Data);
void OLED_WrCmd(unsigned char IIC_Command);
void OLED_Set_Pos(unsigned char x, unsigned char y);
void OLED_Fill(unsigned char bmp_dat);
void OLED_Clear_Line(uint8_t LineNum);
void OLED_CLS(void);
void OLED_Init(void);
extern void OLED_P8x16Str(unsigned char x,unsigned char y,char ch[]);
void OLED_Display(uint8_t line,char *str);
void OLED_delay(unsigned int z);
void IIC_GPIO_Config(void);
void OLED_itoa(unsigned char x,unsigned char y,unsigned char *puls);
void OLED_P8x16StrLen(unsigned char x,unsigned char y,char ch[],unsigned char len);

#endif
```

3) shtxx.c

```
#include "shtxx.h"
// #include "delay.h"
#include "stm32f10x.h"
#include "stm32f10x_rcc.h"
#include "stm32f10x_gpio.h"

#define noACK 0
#define ACK   1

#define STATUS_REG_W 0x06   //000   0011    0
#define STATUS_REG_R 0x07   //000   0011    1
#define MEASURE_TEMP 0x03   //000   0001    1
#define MEASURE_HUMI 0x05   //000   0010    1

/***********************************************************
* Ãû³Æ: IIC_GPIOConfig()
* ¹¦ÄÜ: IIC GPIO³õÊ¼»¯
* ²ÎÊý: ÎÞ
* ·µ»Ø: ÎÞ
* ÐÞ¸Ä:
***********************************************************/
void IIC_GPIOConfig( void )
{
   GPIO_InitTypeDef       GPIO_InitStructure;

   RCC_APB2PeriphClockCmd( SDA_GPIO_CLK | SCL_GPIO_CLK, ENABLE );
   //PB0 PB5
   GPIO_InitStructure.GPIO_Pin = SCL_GPIO_PIN | SDA_GPIO_PIN;
```

```
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
   GPIO_Init( SDA_GPIO_PORT, &GPIO_InitStructure );


//    RCC_APB2PeriphClockCmd( RCC_APB2Periph_AFIO | RCC_APB2Periph_GPIOB, ENAB
LE );
//    GPIO_PinRemapConfig( GPIO_Remap_SWJ_JTAGDisable, ENABLE );    /*Ê¹ ÄÜSWD ½ûÓ
ÃJTAG  Ê¹ PB3 PB4 PA-15Îª IO¿Ú*/
//
//    //Configure SPI1 pins: SCK, MISO and MOSI
//    //Confugure SCK and MOSI pins as Alternate Function Push Pull
//    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
//    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
//    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
//    GPIO_Init( GPIOB, &GPIO_InitStructure );
//
//    GPIOB->BRR  = GPIO_Pin_0;



}
/*************************************************************
* Ãû³ Æ: SDA_OUT()
* ¹ ¦ÄÜ: SDAÒý½ÅéÖÃ³ ÉÍÆÍì Êä³ ö
* ² ÎÊý: ÎÞ
*  • µ»Ø: ÎÞ
* ÐÞ¸ Ä:
*************************************************************/
void SDA_OUT( void )
{
   GPIO_InitTypeDef GPIO_InitStructure;
   GPIO_InitStructure.GPIO_Pin = SDA_GPIO_PIN;
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
   GPIO_Init( SDA_GPIO_PORT, &GPIO_InitStructure );
}
/*************************************************************
* Ãû³ Æ: SDA_IN()
* ¹ ¦ÄÜ: SDAÒý½ÅéÖÃ³ ÉÉÏÀ- ÊäÈë
* ² ÎÊý: ÎÞ
*  • µ»Ø: ÎÞ
* ÐÞ¸ Ä:
*************************************************************/
void SDA_IN( void )
{
   GPIO_InitTypeDef GPIO_InitStructure;
   GPIO_InitStructure.GPIO_Pin = SDA_GPIO_PIN;
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
   GPIO_Init( SDA_GPIO_PORT, &GPIO_InitStructure );
}
/*************************************************************
* Ãû³ Æ: Delay1us()
* ¹ ¦ÄÜ: ÑÓÊ±1usº¯Êý
* ² ÎÊý: ÊäÈë ÐèÒª ÑÓÊ±µÄÎ¢ÃëÊý
```

```
* • µ»Ø: ÎÞ
* ÐÞ, Ä:
* ×¢ÊÍ:
*********************************************************************/
static void Delay1us( uint16_t t )
{
   uint8_t i, j;
   for ( j = t; j > 0; j-- )
      for ( i = 10; i > 0; i-- );
}

/*********************************************************************
* Ãû³ Æ: FS_Init()
* ¹ ¦ÄÜ: ÎÂÊª ¶È´ «¸ ÐÆ ÷ ³ õÊ¼»¯ ¯Êý
* ² ÎÊý: ÎÞ
* • µ»Ø: ÎÞ
* ÐÞ, Ä:
* ×¢ÊÍ:
*********************************************************************/
void FS_Init( void )
{
   IIC_GPIOConfig();
}

/*********************************************************************
* Ãû³ Æ: ConnectionReset()
* ¹ ¦ÄÜ: ÖØÖÃÁ´ ½Ó
* ² ÎÊý: ÎÞ
* • µ»Ø: ÎÞ
* ÐÞ, Ä:
* ×¢ÊÍ:
*********************************************************************/
void ConnectionReset( void )
{
   uint8_t i;
   SDA_OUT();
   Set_IIC_SDA();
   Delay1us( 10 );
   Clr_IIC_SCL();
   Delay1us( 10 );
   for( i = 0; i < 9; i++ )
   {
      Set_IIC_SCL();
      Delay1us( 10 );
      Clr_IIC_SCL();
      Delay1us( 10 );
   }
   TransStart();
}

/*********************************************************************
* Ãû³ Æ: WriteByte()
* ¹ ¦ÄÜ: Ð´ Èë1×Ö½ÚÃüÁî
* ² ÎÊý: ÊäÈë value ÐèÒª Ð´ ÈëµÄÃüÁîµÄÊý¾Ý
* • µ»Ø: ÎÞ
```

```
* ÐÞ, Ä:
* ×¢ÊÍ:
*************************************************************/
static char WriteByte( unsigned char value )
{
   unsigned char i, error = 0;
   SDA_OUT();
   for ( i = 0x80; i > 0; i /= 2 )   //shift bit for masking
   {
      if ( i & value )
         Set_IIC_SDA();         //masking value with i , write to SENSI-BUS
      else
         Clr_IIC_SDA();
      Delay1us( 10 );
      Set_IIC_SCL();                        //clk for SENSI-BUS
      Delay1us( 10 );      //pulswith approx. 5 us
      Clr_IIC_SCL();
      Delay1us( 10 );
   }
   SDA_IN();
   Set_IIC_SDA();                        //release DATA-line
   Delay1us( 10 );
   Set_IIC_SCL();                        //clk #9 for ack
   Delay1us( 10 );
   error = READ_SDA();                   //check ack (DATA will be pulled down by SHT11)
   Clr_IIC_SCL();
   return error;                    //error=1 in case of no acknowledge
}


/*********************************************************************
* Ãû³ Æ: ReadByte()
* ¹ ¦ÄÜ: ¶ÁÈ¡ 1×Ö½ÚÊý¾Ý
* ² ÎÊý: ÊäÈë ack ack=1±íÊ¾ÐèÒª ÏòMMA´ «ÊäÍ£Ö¹ Î»£¬Îª 0Ôò±íÊ¾² »Ðè
*       Òª ÏòMMA´ «ÊäÍ£Ö¹ Î»
*  • µ»Ø: ÎÞ
* ÐÞ, Ä:
* ×¢ÊÍ:
*********************************************************************/
static char ReadByte( unsigned char ack )
{
   unsigned char i, val = 0;

   Set_IIC_SDA();   //release DATA-line
   Delay1us( 10 );
   SDA_IN();
   for ( i = 0x80; i > 0; i /= 2 )   //shift bit for masking
   {
      Set_IIC_SCL();  //clk for SENSI-BUS
      Delay1us( 10 );
      if ( READ_SDA() ) val = ( val | i );  //read bit
      Clr_IIC_SCL();
      Delay1us( 10 );
   }
   SDA_OUT();
   WRITE_SDA( !ack );              //in case of "ack==1" pull down DATA-Line
```

```
   Delay1us( 10 );
   Set_IIC_SCL();   //clk #9 for ack
   Delay1us( 10 );
   Clr_IIC_SCL();
   Delay1us( 10 );
   Set_IIC_SDA();                       //release DATA-line
   Delay1us( 10 );
   return val;
}
/*****************************************************************
* Ãû³ Æ: TransStart()
* ¹¦ÄÜ: ¿ª Ê¼´ «ËÍÊý¾Ý
* ² ÎÊý: ÎÞ
* • µ»Ø: ÎÞ
* ÐÞ¸Ä:
* ×¢ÊÍ:
*****************************************************************/
static void TransStart( void )
{
   SDA_OUT();
   Set_IIC_SDA();
   Delay1us( 10 );
   Clr_IIC_SCL();
   Delay1us( 10 );
   Set_IIC_SCL();
   Delay1us( 10 );
   Clr_IIC_SDA();
   Delay1us( 10 );
   Clr_IIC_SCL();
   Delay1us( 10 );
   Set_IIC_SCL();
   Delay1us( 10 );
   Set_IIC_SDA();
   Delay1us( 10 );
   Clr_IIC_SCL();
   Delay1us( 10 );
}
/*****************************************************************
* Ãû³ Æ: Measure()
* ¹¦ÄÜ: µÃµ½² âÁ¿µÄÖµ
* ² ÎÊý: Êä³ ö p_value Êä³ öµÄÊý¾Ý
      Êä³ ö p_checksum Êä³ öµÄÊý¾ÝÐ£ÑéÖµ
      ÊäÈë Ä£Ê½ ÓÐÎÂ¶ÈÄ£Ê½ ÍÊª ¶ÈÄ£Ê½
* • µ»Ø: ´ íÎóµÄÖµ 0Î ª ÎÞ´ íÎó
* ÐÞ¸Ä:
* ×¢ÊÍ:
*****************************************************************/
uint8_t Measure( unsigned char *p_value, unsigned char *p_checksum, unsigned char mode )
{
   unsigned int error;
   //TransStart();
   ConnectionReset();
   switch( mode )
   {
   case TEMP:
      error = WriteByte( MEASURE_TEMP );
```

```
    break;
  case HUMI:
    error = WriteByte( MEASURE_HUMI );
    break;
  default:
    break;
  }
  SDA_IN();
  error = 0;
  while( error++ < 400000 && READ_SDA() ); //wait until sensor has finished the measurement
  if( READ_SDA() )
  {
    error++;              // or timeout (~2 sec.) is reached
    return 1;
  }
  *( p_value + 1 )  = ReadByte( ACK ); //read the first byte (MSB)
  *( p_value ) = ReadByte( ACK ); //read the second byte (LSB)
  *p_checksum = ReadByte( noACK ); //read checksum
  return 0;
}
/*****************************************************************
* Ãû³ Æ: SHTXX_Cal()
* ¹¦ÄÜ: ×ª»¯µÃµ½µÄÎÂÊª ¶ÈµÄÖµÎª ±ê×¼¼¶µ¥Î»¶Î
* ²ÎÊý: ÊäÈë temp µÃµ½µÄÎÂ¶ÈµÄÖµ
      ÊäÈë p_checksum µÃµ½µÄÊª ¶ÈµÄÖµ
      Êä³ ö f_temp ×ª»¯óµÄÎÂ¶ÈµÄÖµ
      Êä³ ö f_humi ×ª»¯óµÄÎÂ¶ÈµÄÖµ
*  • µ»Ø: ÎÞ
* ÐÞ¸Ä:
* ×¢ÊÍ:
*****************************************************************/
void SHTXX_Cal( uint16_t *temp, uint16_t *humi, float *f_temp, float *f_humi )
{

  const float C1 = -4.0;          // for 12 Bit
  const float C2 = +0.0405;       // for 12 Bit
  const float C3 = -0.0000028;    // for 12 Bit
  const float T1 = +0.01;         // for 14 Bit @ 5V
  const float T2 = +0.00008;      // for 14 Bit @ 5V

  float rh;           // rh:     Humidity [Ticks] 12 Bit
  float t;          // t:      Temperature [Ticks] 14 Bit
  float rh_lin;                 // rh_lin:  Humidity linear
  float rh_true;                // rh_true: Temperature compensated humidity
  float t_C;                    // t_C   :  Temperature

  rh = ( float )( *humi );
  t = ( float )( *temp );

  t_C = t * 0.01 - 40;            //calc. temperature from ticks
  rh_lin = C3 * rh * rh + C2 * rh + C1; //calc. humidity from ticks to [%RH]
  rh_true = ( t_C - 25 ) * ( T1 + T2 * rh ) + rh_lin; //calc. temperature compensated humidity [%R
H]
  if( rh_true > 100 )rh_true = 100; //cut if the value is outside of
  if( rh_true < 0.1 )rh_true = 0.1; //the physical possible range
```

```
    *f_temp = t_C;            //return temperature
    *f_humi = rh_true;          //return humidity[%RH]

}

int SHT11_Read_Data( char* pt, char* ph )
{
    int error = 0;
    float t, h;
    uint16_t humi_val = 0, temp_val = 0;
    char checksum;

    error += Measure( ( unsigned char* ) &temp_val, ( unsigned char* )&checksum,/*TEMP*/0 ); //
measure temperature
    error += Measure( ( unsigned char* ) &humi_val, ( unsigned char* )&checksum,/*HUMI*/1 ); //
measure humidity
    if ( error == 0 )
    {
        SHTXX_Cal( &temp_val, &humi_val, &t, &h );
        *pt = ( int )t;
        *ph = ( int )h;
        return 0;
    }
    return -1;
}
```

4) shtxx.h

```
#ifndef SHTXX_H
#define SHTXX_H


#define SCL_GPIO_PIN GPIO_Pin_1
#define SDA_GPIO_PIN GPIO_Pin_2


#define  SCL_GPIO_PORT        GPIOB
#define  SCL_GPIO_CLK         RCC_APB2Periph_GPIOB


#define  SDA_GPIO_PORT        GPIOB
#define  SDA_GPIO_CLK         RCC_APB2Periph_GPIOB


#define Clr_IIC_SCL()   GPIO_ResetBits(SCL_GPIO_PORT, SCL_GPIO_PIN)
#define Set_IIC_SCL()   GPIO_SetBits(SCL_GPIO_PORT, SCL_GPIO_PIN)

#define Set_IIC_SDA()   GPIO_SetBits(SDA_GPIO_PORT, SDA_GPIO_PIN)
#define Clr_IIC_SDA()   GPIO_ResetBits(SDA_GPIO_PORT, SDA_GPIO_PIN)
#define READ_SDA()      GPIO_ReadInputDataBit(SDA_GPIO_PORT, SDA_GPIO_PIN)
#define WRITE_SDA(x)    GPIO_WriteBit(SDA_GPIO_PORT, SDA_GPIO_PIN, x)


enum {TEMP,HUMI};

void IIC_GPIOConfig(void);
void SDA_OUT(void);
void SDA_IN(void);
```

```
void SHTXX_Init(void);
void FS_Init(void);
static char WriteByte(unsigned char value);
static char ReadByte(unsigned char ack);
void ConnectionReset(void);
static void TransStart(void);
unsigned char Measure(unsigned char *p_value, unsigned char *p_checkum, unsigned char mode);
void SHTXX_Cal(unsigned short *temp, unsigned short *humi, float *f_temp, float *f_humi );
int SHT11_Read_Data(char* pt, char* ph);


#endif
```

5) main.c

```
#include <stdio.h>
#include <math.h>
#include "stm32f10x.h"
#include "usart.h"
#include "delay.h"
#include "shtxx.h"
#include "LQ12864.h"

#define SHTXX_Init() IIC_GPIOConfig()

int main(void)
{
  uint8_t error, checksum;
  uint16_t humi_val, temp_val;
  char temp[16] = {0},humi[16] = {0};
  float f_humi, f_temp;

  delay_init(72);
  uart1_init();
  OLED_Init();
  SHTXX_Init();
  OLED_Display(OLED_Line_2, "HumiTemp Exp.");
  printf("Stm32 Sensors example start !\r\n");

  while(1)
  {
    error = 0;
    error += Measure((unsigned char*) &temp_val,&checksum,TEMP);  //测量温度
    error += Measure((unsigned char*) &humi_val,&checksum,HUMI);  //测量湿度

    if (error == 0)
    {
      //通过公式将读取的温湿度的值转换成可识别的值
      SHTXX_Cal(&temp_val, &humi_val, &f_temp, &f_humi);
      sprintf(temp, "Temp = %.1fC", f_temp);
      sprintf(humi, "Humi = %.1f%%", f_humi);

      OLED_Clear_Line(OLED_Line_3);
      OLED_Clear_Line(OLED_Line_4);
      OLED_Display(OLED_Line_3, temp);   //OLED 屏打印温湿度信息
      OLED_Display(OLED_Line_4, humi);
```

```
      printf("Temp = %.1f℃, Humi = %.1f%%\r\n", f_temp, f_humi);  //串口打印温湿度信息
    }
    delay_ms(1000);
  }
}
```